# Aircraft Integrated Management Sandbox (AIMS) Block 0: Robust Inner Loop Aircraft Control in JSBSim

Alexander Bowman

August 2026

## 1 Introduction

Since the widespread adoption of fly-by-wire systems, aircraft control has been a field of ever increasing importance. Using control systems, fighters can be made more maneuverable, passenger aircraft made more comfortable, and passive stability requirements are no longer so strenuous. Additionally advances in aircraft control have enabled the development of advanced and autonomous unmanned aerial vehicles. With these trends in mind, the Aircraft Integrated Management Sandbox (AIMS) project was undertaken with the goal to demonstrate modern aircraft control techniques implemented in a realistic environment. This software was designed to align with the open-architecture philosophy described by standards such as the United States Air Force's Autonomy Government Reference Architecture (A-GRA).

In AIMS JSBSim is used as the flight dynamics model (FDM), as it is a lightweight and accurate [1] FDM and is widely used in research, notably in DARPA's AlphaDogfight Trials [2]. Test were performed using the F-15 aircraft model included with JSBSim. The AIMS Tuning Manager allows the user to create a state-space lateral and longitudinal linearizations of the aircraft at a given airspeed, altitude, and weight directly from the JSBSim model and to tune the controllers based on those state-space models. These gains are saved to a `.json` file and are scheduled using the Mach number, dynamic pressure, and weight to determine the gains for a given flight condition. These gains are passed back to the flight execution loop and the controller output is applied to the aircraft in JSBSim. The commands for the controller can be set using the AIMS Flight Planner, which saves flight plans to a `.json` file and can generate a matrix of state command vectors from a loaded flight plan. Timestamped state output from the execution loop can be saved as a CSV and plotted by the included output visualizer or could be read by a custom program.

AIMS Block 0 is intended to be a minimum viable product to demonstrate inner loop aircraft control and gain scheduling infrastructure. In its current form, AIMS uses full state feedback and perfect full state observation. Future development is planned to focus on outer loop control, observers, Kalman filtering, and potentially mission autonomy.

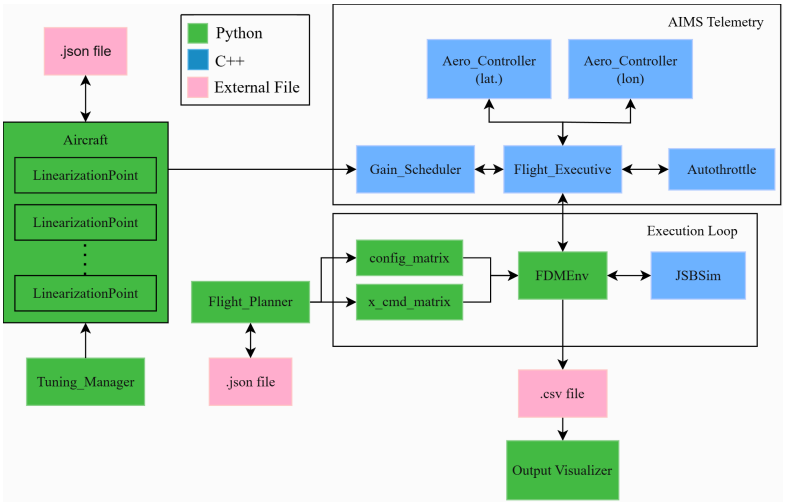


Figure 1: AIMS Block 0 Software Architecture. This architecture separates the offline setup and control law tuning (left) from the online control law execution (top right), which is too is separated from the FDM orchestration and output visualization (bottom right).

## 2 Software Architecture

AIMS is currently written in a combination of Python 3.11.15 and C++23. Python is used for offline code to interface with the user using a command line interface, orchestration between the controllers and the FDM, output visualization, and other tasks that would not be executed on an aircraft flight computer. C++ is used for online tasks such as the controllers, gain scheduler, and other tasks that would be performed by an aircraft flight computer. This architecture was used to allow the ease of development of Python for non-online portions of the software, while flight-critical code is written in C++, reflecting common industry practice. C++ code is compiled to interface with Python using Pybind11.

Safe C++ coding practices, such as declaring constant variables with `const`, as well as avoiding recursion and dynamic memory allocation can be used and were partially implemented to reflect industry practice and ensure predictable and safe operation. Currently, exception handling is used to make debugging more straightforward, and the Eigen library is used for linear algebra although the library is not certified for real-time or safety-critical systems. Additionally, recursive C++ code is used to generate the k-d trees used in gain scheduling and dynamically sized vectors and matrices from the Eigen library are widely used. The k-d tree building code may be moved to offline execution in a future update, eliminating this online recursive code. Since the design goal of AIMS is to be a highly flexible aircraft controller test software rather than a flown flight controller, at this time the flexibility afforded by dynamically sized vectors outweighs the non-deterministic execution due to Eigen dynamically sizing matrices.

The Tuning Manager is used to linearize the aircraft at a specified flight condition, to specify which variables will have an error integrator applied, and allows the user to tune a pair of lateral and longitudinal linear quadratic regulator (LQR) controllers for the condition. Currently it is assumed that the lateral and longitudinal systems are fully decoupled.

Gains and stability metrics are calculated using the Python Control Library [3]. To keep track of which states have an error integrator applied, an integrator map is created, which is by default an integer vector of zeros where the index of the state with the first error integrator applied is set to one, the second integrator state index set to two, etc. The flight conditions, trim conditions, controller gains, state-space matrices, and stability metrics for the specified altitude, airspeed, weight, and gear/flap positions are stored in the LinearizationPoint class. A list of LinearizationPoints is stored in the Aircraft object. The Aircraft class also contains information about the aircraft itself (aircraft type, actuator model parameters, integrator maps, etc.). An Aircraft object can be read or saved to a .json file using the method `from_json` and the function `export_to_json`; the aircraft data can also be passed to the gain scheduler using the function `get_scheduling_data`.

The gain scheduler takes input from the aircraft object and constructs a set of k-d trees which is used to generate an EnclosingCell comprised of four points that enclose the current flight condition in the $q$, $M$, $W$ space. The controller gains and trim conditions are found by using Barycentric interpolation between the four points. The gain scheduler will be discussed in greater detail in section 4.

The FDM Environment interfaces between the loaded flight plan and the Flight Executive while running the execution loop. The JSBSim Python library was used to implement the FDM. AIMS uses knots, feet, radians, and seconds as internal state units and control inputs are normalized. The FDM environment interfaces with the controllers via the Flight Executive. Output from the simulation is saved to a `.csv` file and can be read and plotted using the included output visualizer.

# 3 Tuning Manager

The Tuning Manager linearizes the aircraft at given flight airspeed, altitudes, and weights and allows the user to tune an LQR lateral and longitudinal controller. Currently the longitudinal (1) and lateral state (2) vectors are fixed.

$$x_{lon} = \begin{bmatrix} \alpha \\ q \end{bmatrix} \tag{1}$$

$$x_{lat} = \begin{bmatrix} \beta \\ p \\ r \end{bmatrix} \tag{2}$$

The Python Control Library is used to solve the Algebraic Riccati Equation (ARE).

$$PA + A^T P - PBR^{-1}B^T P + Q = 0 \tag{3}$$

$$K = -R^{-1}B^T P \tag{4}$$

The control vector (5) is fixed as well, with control inputs specifying the normalized throttle $\delta_t$, normalized elevator command $\delta_e$, normalized aileron command $\delta_a$, and normalized rudder command $\delta_r$. While this may eventually pose issues with aircraft with non-traditional control layouts such as tailless aircraft, aircraft with canards, or aircraft with thrust vector control (TVC), there are currently no known high-fidelity JSBSim models of flying wings or aircraft with canards, and despite the F-22 using TVC the JSBSim F-22 model automatically converts elevator commands into an elevator and TVC command.

$$u = \begin{bmatrix} \delta_t \\ \delta_e \\ \delta_a \\ \delta_r \end{bmatrix} \tag{5}$$

The JSBSim F-15 model uses a first order actuator model. A first order actuator model can be described using the current actuator position $\delta$, the commanded actuator position $\delta_{cmd}$, and a rate limit constant $\tau$:

$$\dot{\delta} = \frac{1}{\tau}\left(\delta_{cmd} - \delta\right) \tag{6}$$

The aircraft linearized model, actuator model, and the controller observability matrix $C$ are used to develop a full state for the lateral and longitudinal states. $C$ can be recreated using an integrator map as described in section 5.

$$\dot{z} = Az + Bu_{cmd} \tag{7}$$

$$\begin{bmatrix} e \\ \dot{x}_{aero} \\ \dot{x}_{act} \end{bmatrix} = \begin{bmatrix} 0 & C & 0 \\ 0 & A_{aero} & B_{aero} \\ 0 & 0 & A_{act} \end{bmatrix} \begin{bmatrix} \int e \\ x_{aero} \\ x_{act} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ B_{act} \end{bmatrix} u_{cmd} \tag{8}$$

# 4 Gain Scheduling

The aircraft state-space matrices vary nonlinearly with the aircraft operating conditions. To capture these variations, the controller gains are scheduled using dynamic pressure $q$, Mach number $M$, and fuel weight $W$. Aircraft body forces are a function of dynamic pressure, Mach number captures the impacts of compressibility and shocks on the aerodynamic model, and the weight of the fuel accounts for the changes in inertial properties as fuel is consumed.

Separate k-d trees are generated for each flap and gear configuration combination. Gear position is bucketed into 0 (gear up) and 1 (gear down). Flaps are similarly bucketed into normalized positions 0 (flaps up), 0.10, 0.20, 0.30, 0.50, 0.75, and 1.00 (flaps down).

Three-dimensional interpolation is performed using Barycentric interpolation. Barycentric interpolation was used so that unstructured $M$-$q$-$W$ sets could be used. The gain scheduler finds the four linearization points which enclose the operating condition on the Mach number-dynamic pressure-weight plot. Barycentric interpolation was used

because the distribution of linearization points is expected to be unstructured. To perform Barycentric interpolation, the Barycentric coordinates of the aircraft operating condition, $(\lambda_1, \lambda_2, \lambda_3, \lambda_4)$, are four points which enclose the operating condition where point 1 is the nearest and 4 the furthest and have the normalized coordinates $(1, 0, 0, 0)$, $(0, 1, 0, 0)$, $(0, 0, 1, 0)$, and $(0, 0, 0, 1)$ respectively. These Barycentric coordinates are used to find the interpolated gains.

$$K_{interp} = \lambda_1 K_1 + \lambda_2 K_2 + \lambda_3 K_3 + \lambda_4 K_4 \tag{9}$$

The trim state, trim control, or any other value which is derived from the Aircraft LinearizationPoints is similarly found by multiplying the Barycentric weights by the value associated with the point's weight.

If the aircraft remains within the EnclosingCell from the previous iteration of the execution loop, the gain scheduler only runs the interpolation code and if the flight conditions remain within a set percent of the conditions at which the last time the gain scheduler was run (for example during a steady cruise) the gain scheduler returns the previous results. Reusing the EnclosingCell across multiple timesteps reduces the number of expensive k-d tree searches needed. For conditions outside of the flight envelope, the nearest set of gains on the Mach number-dynamic pressure plot is used.

# 5 Aerodynamic Controller

Both the lateral and longitudinal aerodynamic controller are instances of the same underlying AeroController class.

When initalizing the AeroController, an integrator map of the same length of the state vector (including actuator states) is given. If a state does not have its error tracked, its index on the integrator map is set to zero, the index on the integrator map for the first state with error tracking is set to one, and each further error tracked state increases by one.

The AeroController is fairly straight forward; after the Flight Executive breaks down the passed `x_cmd` and `x` into their lateral and longitudinal components, gets the estimated trim states, trim control, and gains from the gain scheduler, the AeroController updates the error integrator vector, constructs the augmented state vector $z$ (10), applies the control law (11), and applies position limiters to the control output.

$$z = \begin{bmatrix} \int e_1 \\ \vdots \\ \int e_n \\ x \end{bmatrix} \tag{10}$$

$$u = -Kz \tag{11}$$

In testing high-frequency oscillations were observed in the actuator positions. These oscillations are likely caused by the discrete time sampling process. Decreasing the size of the JSBSim time step marginally improved but did not eliminate the oscillation and significantly increased the number of JSBSim steps performed. Rather than decrease

the JSBSim step size to an impractically small number, to eliminate the issue a low-pass filter was implemented:

$$\delta_{\text{filter}} = \alpha\delta_{\text{unfiltered}} + (1-\alpha)\delta_{\text{filter, prev}} \tag{12}$$

# 6 Auto-throttle

The AIMS auto-throttle is a fairly simple proportional-integral-derivative (PID) controller. A PID controller was used for this system since throttle control is a single-input single-output (SISO) system for which modeling the engine would be difficult and unnecessary. The PID controller is governed by the equation with proportional gain $K_p$, integral gain $K_i$, derivative gain $K_d$, and error $e$:

$$u_{throttle} = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \tag{13}$$

Since afterburner engages for throttle commands above 0.99, greatly increasing fuel consumption, a boolean was added to the auto-throttle controller to toggle afterburner. Anti-windup measures were implemented by keeping the integral error from increasing when the current air speed is less than the commanded air speed while the throttle is at the maximum position or while the current air speed is more than the commanded air speed and the engines are commanded to idle.

The auto-throttle tracks gradual increases and decreases in commanded airspeed quite well, and can hold constant air speeds steady. However, as the performance of the engines have limits, the auto-throttle may not perform as commanded tracking a large acceleration or when the aircraft is subjected to high drag, and since the behavior of deceleration differs considerably from acceleration, tends to overshoot a sudden deceleration.

# 7 Results

To test AIMS, the JSBSim F-15 was linearized with flap and gear up at a total of seven different combinations of airspeed, altitude, and fuel weight. At each point, the longitudinal controller was tuned for an overshoot of near 10% and 90% rise time near a quarter of a second. Gain margin and phase margin requirements were never in danger of being exceeded (gain margin of over 20 dB and phase margin of over 60 degrees was typical), nor were peak sensitivity $\overline{\sigma}(S_u)$ or peak co-sensitivity $\overline{\sigma}(T_u)$ ever of concern (values near 1.00 to 1.10 were typical). For the lateral controller, overshoot of 15% was targeted as was a 90% rise time of 0.10 to 0.25 seconds. Gain margin, phase margin, peak sensitivity and peak co-sensitivity values were similar for the lateral controller.

A thirty-second flight plan was created for the aircraft starting at an altitude of 20,000 ft with 5,000 lbs of fuel at a constant airspeed of 400 kts. In this flight plan, there would be a pitch rate command doublet of five degrees per second, a five second rest, and a roll rate command doublet of five degrees per second. The autothrottle was

| Point ID | Mach | $q$ (Pa) | Fuel (lb) | Alt (ft) | $V$ (kts) |
|---|---|---|---|---|---|
| 0 | 0.65 | 13800 | 5000 | 20000 | 400 |
| 1 | 0.65 | 13800 | 500 | 20000 | 400 |
| 2 | 0.65 | 13800 | 10000 | 20000 | 400 |
| 3 | 0.81 | 21600 | 5000 | 20000 | 500 |
| 4 | 0.49 | 7800 | 5000 | 20000 | 300 |
| 5 | 0.68 | 9700 | 5000 | 30000 | 400 |
| 6 | 0.63 | 19200 | 5000 | 10000 | 400 |

Table 1: F-15 test linearization points.

run at 10 Hz, the aerodynamic controllers were run at 100 Hz, and the JSBSim environment was run at 250 Hz. This flight plan was saved as a `.json` file and loaded into the FDM Environment. The output was saved as a `.csv` file and plotted using the Output Visualizer as seen in figure 2.

As seen in figure 2 both the pitch rate and roll rate commands have rise times and overshoots much less than the idealized closed-loop analysis predicted. The aircraft was controlled using an LQR controller which assumes that the system is linear time-invariant (LTI), which this system is not. During the pitch rate doublet command despite the engines operating near full throttle, the airspeed continues to decrease through the positive portion of the doublet, although the gain scheduler should be able to return interpolated gains to mitigate these effects. It is also possible that this under-performance could be caused by the nonlinearity of the flight dynamics themselves, such as nonlinear drag, but if this were the sole culprit the roll rate doublet would not have similarly undershot and rose slowly. A more likely explanation is the flight dynamics linearization script being poorly configured. This issue could be addressed by rewriting the linearization script, intentionally overtuning the aerodynamic controllers, or manually setting the state space matrices. At present ideas on how to rework the linearization script are lacking and the ability to manually set the state space matrices has not yet been implemented, leaving intentional overtuning as the only currently viable method.

# 8 Conclusion

AIMS successfully demonstrated LQR controller tuning, gain scheduling, and closed-loop control for the fast lateral and longitudinal flight modes as well PID control of the throttle system in JSBSim. This is a strong foundation on which further control and autonomy work can be demonstrated. Future projects may include Kalman filters, model-reference adaptive control (MRAC), and/or mission autonomy.

Despite the success of AIMS it is not a software without fault. There are significant discrepancies between the closed-loop analysis and the actual response in the execution loop, for example, control gains which claim near 10% overshoot may instead undershoot and have a much longer rise time when run in the execution loop. This could be addressed by a better flight dynamics linearization process or by allowing the user to manually set the state matrices at a given flight condition. Being the author's first

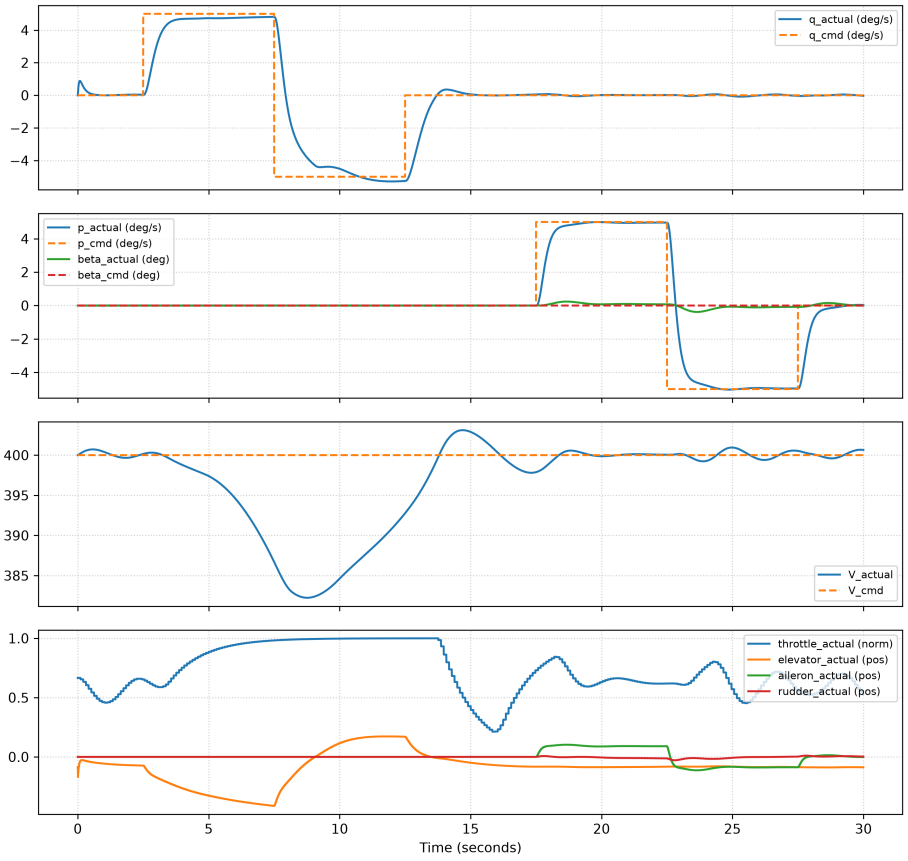


Figure 2: Pitch and roll rate command doublet response

project of such scale and scope, much of the Python code is organized in such a way that may make a transition from a command line interface to a graphical user interface cumbersome and other expansions difficult. Large portions of the code base may significantly benefit from refactoring.

# References


[1] Murri, D. G., Jackson, E. B., and Shelton, R. O., "Check-Cases for Verification of 6-Degree-of-Freedom Flight Vehicle Simulations," Tech. rep., National Aeronautics and Space Administration, 2015.

[2] Pope, A. P., Ide, J. S., Mićović, D., Diaz, H., Twedt, J. C., Alcedo, K., Walker, T. T., Rosenbluth, D., Ritholtz, L., and II, D. J., "Hierarchical Reinforcement Learning for Air Combat at DARPA's AlphaDogfight Trials," *IEEE Transactions on Artificial Intelligence*, Vol. 4, No. 6, November 2022, pp. 1371–1385.

[3] Fuller, S., Greiner, B., Moore, J., Murray, R., van Paassen, R., and Yorke, R., "The Python Control Systems Library (python-control)," *60th IEEE Conference on Decision and Control (CDC)*, IEEE, 2021, pp. 4875–4881.